Cool Code

What goes into making processors that (literally) don't burn holes in your pockets?

Nimish Chandiramani

Our lives are now practically infested with mobile devices. Whether it's your old cell phone, a shiny new PDA, or the upcoming Ultra-mobile PCs (UMPC), the quest now is to pack in as much PC-like functionality into these gadgets as possible. The challenges come in when you realise that the processors for these devices don't have the luxury of living in a capacious cabinet with monster-sized heat-sinks to keep them cool—with the walls closing in on them, they must deal with computation in a completely different way.

Nearly 75 per cent of all embedded processors are based on the ARM Architecture (Advanced RISC Machine, or Acorn RISC Machine), because it meets such requirements beautifully—it's half the size of an old 486 CPU, is almost as powerful (with the potential for more), doesn't heat up even under load, and consumes but a smidgen of battery power. How does it do it, and why don't we have such features in our desktop processors?

History Lesson, I: Less Is More

Today's mobile processors owe their architecture to a division in processor design philosophy that started somewhere in the 1970s. Flashback (in black-and-white, if you like) to a dark age where no compilers existed, and processors were hand-coded in their own assembly language instructions. These instruction sets were quite simplistic, and something as simple as comparing two numbers could take ten instructions to execute. Imagine trying to write an operating system with that! But that was just half the problem.

When you gave an instruction to a processor, you do so as a hexadecimal *op-code*, which was stored in a register on the CPU itself before being executed. This register (a *pipeline* of many registers in today's processors) is an expensive resource to waste, and the number of instructions required for the simplest operations made that painfully obvious—more so when processors were getting powerful enough to do so much more. The goal, then, was to create instructions that were the same length (in bits, that is), but led to the execution of many *micro-operations*. For example, the 8086's JPE (Jump If Equal) instruction loads two numbers from the system's memory, subtracts them, uses that result to determine whether they're equal or not, and then jump to a new instruction if they are indeed equal. Contrast this with its predecessor, the 8085, with but the humble JMP instruction, which made the processor jump to the specified memory location.

This was the birth of Complex Instruction Set Computing (CISC)—simple instructions that performed complex tasks. It didn't waste

Imaging Praveen Warhokar
Photograph Sandip Patil